

## Введение

Производственная практика проходила в Московском университете имени С.Ю. Витте на кафедре информационных систем. В ходе практики рассматривались методы обработки научных PDF-документов и возможности использования нейросетевых моделей при анализе математических текстов.

Основной задачей работы являлась разработка программы на языке Python для автоматического извлечения теорем и лемм из научных статей. Дополнительно программа должна была определять название статьи и авторов при помощи локальной нейросети, подключённой через Ollama.

Во время выполнения работы изучались особенности структуры PDF-документов, способы очистки текста после извлечения, а также методы поиска математических утверждений. Особое внимание уделялось сохранению формул и корректной обработке научных текстов.

Целью практики являлось получение практических навыков разработки программ для обработки научной информации и подготовки структурированного датасета.

Для достижения поставленной цели были поставлены следующие задачи:

- изучение библиотек Python для работы с PDF-документами;
- разработка программы для извлечения текста из научных статей;
- реализация поиска теорем и лемм в тексте;
- очистка и обработка извлечённых данных;
- подключение локальной нейросети через Ollama;
- автоматическое определение названия статьи и авторов;
- формирование итогового JSON-файла с результатами обработки.

В процессе работы использовались библиотека PyMuPDF для чтения PDF-документов, библиотека requests для отправки запросов к локальной модели, а также модель Qwen, запущенная через Ollama.

## 2. Анализ предметной области

## **2.1. Выбор и настройка среды разработки Python**

Для разработки программы был выбран язык Python, так как он хорошо подходит для обработки текста, работы с файлами и автоматизации различных задач. Кроме этого, Python имеет большое количество библиотек для анализа документов и взаимодействия с нейросетевыми моделями.

Разработка программы выполнялась в среде PyCharm. Данная среда разработки позволяет удобно работать с проектом, подключать библиотеки, запускать программу и проверять результаты обработки данных.

В ходе работы были установлены библиотеки `fitz`, `requests`, `json`, `re` и `os`. Библиотека `fitz` из `PyMuPDF` использовалась для чтения PDF-документов и получения текста со страниц. Библиотека `requests` применялась для отправки запросов к локальной нейросети через Ollama. Модуль `re` использовался для очистки текста и поиска теорем и лемм при помощи регулярных выражений. Модуль `json` применялся для формирования итогового датасета.

Также была настроена рабочая папка проекта, содержащая PDF-файлы научных статей, основной файл программы и итоговый JSON-файл с результатами обработки. После настройки среды была выполнена проверка корректности чтения PDF-документов и подключения локальной модели Qwen через Ollama.

## **2.2. Анализ применения нейросетей для обработки научных текстов**

В настоящее время нейросети активно применяются для анализа текстовой информации. Особенно это актуально при обработке научных публикаций, так как такие документы имеют сложную структуру, содержат специальные символы, формулы и большое количество терминов.

При помощи языковых моделей можно автоматически извлекать информацию о статье, находить нужные текстовые фрагменты и анализировать содержимое документа. Это позволяет значительно сократить время обработки большого количества научных публикаций.

В данной работе нейросеть использовалась для определения названия статьи и авторов по содержимому первой страницы PDF-документа. Для этого была выбрана модель Qwen, запущенная локально через Ollama.

Подключение модели выполнялось через HTTP API при помощи библиотеки requests. Программа отправляла текст первой страницы PDF-документа в модель, после чего получала структурированный ответ с названием статьи и авторами.

Использование локальной модели позволило автоматизировать часть обработки научных текстов и упростить формирование итогового датасета.

## **2.3. Анализ структуры научных PDF-документов**

Научные PDF-документы имеют сложную структуру и отличаются от обычного текста. В статьях присутствуют формулы, специальные математические символы, сноски, различные шрифты и элементы оформления. Из-за этого при извлечении текста могут появляться ошибки, лишние пробелы и переносы строк.

Большинство рассмотренных PDF-документов содержали название статьи, авторов, аннотацию, основной текст, теоремы, леммы и математические выражения. При этом структура статей в разных документах отличалась. В некоторых файлах автор располагался над названием статьи, а в некоторых - под ним. Также в документах встречались сноски и дополнительные символы, которые мешали корректному определению названия статьи.

Для чтения PDF-документов использовалась библиотека PyMuPDF. Она позволяет открывать PDF-файлы, проходить по страницам документа и извлекать текстовое содержимое.

Во время анализа структуры документов было установлено, что после извлечения текста появляются следующие проблемы:

- переносы строк внутри предложений;
- лишние пробелы;
- слипшиеся символы;
- неправильное отображение математических формул;
- появление номеров сносок внутри названий статей.

Для решения этих проблем в программе была реализована дополнительная очистка текста при помощи регулярных выражений.

## **2.4. Анализ методов поиска теорем и лемм**

Одной из основных задач программы являлся поиск теорем и лемм в тексте научных статей. Для решения этой задачи был выбран подход на основе регулярных выражений.

Во многих математических статьях формулировки начинаются со слов «Теорема» или «Лемма», после которых указывается номер и текст утверждения. Это позволило реализовать поиск при помощи шаблонов регулярных выражений.

Программа анализирует текст страницы и ищет конструкции, начинающиеся со слов «Теорема» или «Лемма». После нахождения совпадения формируется отдельный текстовый фрагмент, содержащий найденное утверждение.

Дополнительно программа сохраняет начальный и конечный индекс найденного фрагмента. Это позволяет определить положение теоремы или леммы внутри текста документа.

Также в ходе работы было установлено, что при обработке математических текстов важно сохранять формулы и специальные символы в исходном виде. Поэтому было принято решение отказаться от сложной нормализации текста, так как она могла исказить математические выражения.

## **2.5. Вывод по разделу**

В данном разделе были рассмотрены особенности обработки научных PDF-документов и методы анализа математических текстов. Также были изучены инструменты Python, необходимые для реализации программы.

Проведённый анализ позволил определить основные проблемы при извлечении текста из PDF-файлов и выбрать подход для поиска теорем и лемм при помощи регулярных выражений. Кроме этого, был изучен способ подключения локальной нейросети через Ollama для автоматического определения названия статьи и авторов.

## 3. Разработка системы подготовки датасета

### 3.1. Разработка структуры программы

Разработка программы началась с создания основной структуры проекта. Все действия по обработке PDF-документов были разделены на отдельные функции. Такой подход позволил сделать программу более понятной и упростил дальнейшую доработку.

В проекте используются отдельные функции для:

- очистки текста;
- очистки строк с метаданными;
- обработки авторов статьи;
- извлечения информации о статье;
- поиска теорем и лемм;
- формирования итогового JSON-файла.

Основной файл программы содержит последовательную обработку PDF-документов. Сначала программа получает список файлов из рабочей папки, затем открывает каждый PDF-документ и выполняет его анализ.

Для хранения итоговых данных используется список `all_results`. В него добавляется информация о найденных теоремах и леммах, название статьи, авторы, номер страницы и индексы фрагмента.

```
all_results = []
```

После завершения обработки всех документов результаты сохраняются в файл `result.json`.

Такая структура программы была выбрана для того, чтобы логика обработки была разделена на небольшие независимые части. Это упростило тестирование программы и исправление ошибок во время разработки.

## 3.2. Извлечение и обработка текста из PDF-документов

### 3.2.1. Получение и чтение PDF-файлов

На данном этапе была реализована загрузка PDF-файлов из рабочей папки проекта. Для этого используется папка `pdf_files`, в которую заранее помещаются научные статьи для обработки.

Программа проходит по содержимому папки и выбирает только файлы с расширением `.pdf`.

После получения имени файла формируется полный путь к PDF-документу.

```
for file in os.listdir(folder):  
    if file.endswith(".pdf"):  
        path = os.path.join(folder, file)
```

Для открытия PDF-файла используется библиотека `PyMuPDF`.

```
doc = fitz.open(path)
```

После открытия документа программа проходит по всем страницам статьи и получает текст каждой страницы отдельно.

```
for page_num, page in enumerate(doc):  
    text = page.get_text()
```

Такой подход позволяет обрабатывать большие PDF-документы постранично и выполнять поиск теорем и лемм внутри каждой страницы отдельно.

### 3.2.2. Очистка текста

После извлечения текста из PDF-документа выполняется его очистка. Это необходимо, так как после чтения PDF-файлов часто появляются переносы строк, лишние пробелы и некорректное разделение слов.

Для очистки текста используется функция `clean_text()`.

```
def clean_text(text): 2 usages  nesquikduo
    text = re.sub(pattern: r'(\w)-\s+(\w)', repl: r'\1\2', text)
    text = text.replace(__old: "\n", __new: " ")
    text = text.replace(__old: "\t", __new: " ")
    text = re.sub(pattern: r"\s+", repl: " ", text)
    return text.strip()
```

В данной функции сначала исправляются переносы слов внутри текста при помощи регулярного выражения.

```
text = re.sub(r'(\w)-\s+(\w)', r'\1\2', text)
```

После этого удаляются переносы строк и табуляции.

```
text = text.replace("\n", " ")
text = text.replace("\t", " ")
```

Затем при помощи регулярного выражения убираются повторяющиеся пробелы.

```
text = re.sub(r"\s+", " ", text)
```

В результате текст приводится к более удобному виду для дальнейшей обработки.

Дополнительно в программе используется функция `clean_metadata_line()`. Она применяется при обработке названия статьи и авторов.

```
def clean_metadata_line(text): 1 usage  nesquikduo *
    text = clean_text(text)
    text = re.sub(pattern: r'([А-Яа-яЁёА-За-з])\d+$', repl: r'\1', text)
    return text.strip()
```

В данной функции удаляются лишние символы и номера сносок, которые могут появляться внутри названия статьи после извлечения текста из PDF-документа.

### 3.2.3. Формирование текстовых фрагментов

После очистки текста программа выполняет поиск теорем и лемм. Для этого используется функция `extract_blocks()`.

```
def extract_blocks(text):
    text = clean_text(text)

    pattern = (
        r'(      r'(Теорема|Лемма)      r'(?:\s\d+(?:\.d+)*)?      r'\s*[\.\—:]'      r'\s*'
        r'.*?'      r')'      r'(?='
        r'\s+(?:Теорема|Лемма|Доказательство|Следствие|Определение|Замечание|Приме
        p)'      r'(?:\s\d+(?:\.d+)*)?      r'\s*[\.\—:]'      r'|'$      r')' )

    results = []

    for match in re.finditer(pattern, text, flags=re.IGNORECASE):
        block_text = match.group(1).strip()
        block_type_word = match.group(2).lower()

        if "теорема" in block_type_word:
            block_type = "theorem"
        elif "лемма" in block_type_word:
            block_type = "lemma"
        else:
            continue
        results.append({
            "type": block_type,
            "text": block_text,
            "start_index": match.start(),
            "end_index": match.end()
        })

    return results
```

В данной функции применяется регулярное выражение, которое ищет конструкции, начинающиеся со слов «Теорема» или «Лемма». Такой подход позволяет автоматически находить математические утверждения в тексте статьи.



```
pattern = (
    r'( r'(Теорема|Лемма) r'(?:\s+\d+(?:\.\d+)*)? r'\s*[\.—:]' r'\s*' r'.*?' r')'
    r'(?='
    r'\s+(?:Теорема|Лемма|Доказательство|Следствие|Определение|Замечание|Приме
    p)' r'(?:\s+\d+(?:\.\d+)*)? r'\s*[\.—:]' r'|$' r'))
```

После нахождения совпадения программа формирует отдельный текстовый фрагмент. Для каждого найденного блока дополнительно сохраняются начальный и конечный индексы.

```
results.append({
    "type": block_type,
    "text": block_text,
    "start_index": match.start(),
    "end_index": match.end()
})
```

Тип найденного блока сохраняется отдельно. Это позволяет различать теоремы и леммы в итоговом датасете.

Также в программе реализована дополнительная обработка найденных фрагментов. Она нужна для удаления лишнего текста, который может появляться после формулировки теоремы или леммы.

### 3.3. Реализация поиска теорем и лемм

Основной задачей программы является автоматический поиск теорем и лемм в математических статьях. Для решения этой задачи использовались регулярные выражения.

Во многих научных статьях формулировки начинаются со слов «Теорема» или «Лемма», после которых указывается номер и текст утверждения. Это позволило использовать единый шаблон поиска.

Программа анализирует текст страницы и проверяет наличие совпадений с регулярным выражением.

```
for match in re.finditer(pattern, text, flags=re.IGNORECASE):
```

После нахождения совпадения формируется отдельный объект с информацией о найденном фрагменте.

```
if "теорема" in block_type_word:  
    block_type = "theorem"  
  
elif "лемма" in block_type_word:  
    block_type = "lemma"
```

В итоговый JSON-файл сохраняются:

- имя PDF-файла;
- название статьи;
- авторы;
- номер страницы;
- тип найденного фрагмента;
- текст теоремы или леммы;
- начальный индекс;
- конечный индекс.

Такой подход позволил автоматически формировать структурированный датасет с математическими утверждениями из научных публикаций.

### 3.4. Использование нейросети для анализа текста

нейросеть в программе используется для определения названия статьи и авторов. Это необходимо, так как структура PDF-документов отличается, и получить такую информацию обычными регулярными выражениями достаточно сложно.

В программе используется локальная языковая модель Qwen, запущенная через Ollama. Подключение к модели выполняется при помощи HTTP API.

В качестве модели была выбрана qwen2.5:3b.

```
OLLAMA_URL = "http://localhost:11434/api/generate"  
OLLAMA_MODEL = "qwen2.5:3b"
```

Для отправки запросов используется библиотека requests.

```

response = requests.post(
    OLLAMA_URL,
    json={
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False,
        "format": "json",
    }
)

```

Программа отправляет в модель строки первой страницы PDF-документа, после чего получает JSON-ответ с названием статьи и авторами.

Использование нейросети позволило автоматизировать обработку метаданных статьи и сократить количество ручной работы при подготовке датасета.

### 3.4.1. Определение названия статьи и авторов

После подключения нейросети была реализована автоматическая обработка первой страницы PDF-документа. Это необходимо для того, чтобы программа самостоятельно определяла название статьи и авторов перед сохранением данных в итоговый JSON-файл.

Для решения этой задачи используется функция `extract_article_metadata()`.

```

def extract_article_metadata(doc):
    page = doc[0]
    page_dict = page.get_text("dict")

    lines = []

    for block in page_dict["blocks"]:
        if block.get("type") != 0:
            continue
        for line in block["lines"]:
            text = ""
            max_size = 0
            x0, y0, x1, y1 = line["bbox"]

            for span in line["spans"]:
                text += span["text"]
                max_size = max(max_size, span["size"])

            text = clean_metadata_line(text)

```

```

    if text:
        lines.append({
            "text": text,
            "font_size": round(max_size, 2),
            "x": round(x0, 2),
            "y": round(y0, 2)
        })

    lines = sorted(lines, key=lambda item: (item["y"], item["x"]))

    useful_lines = []

```

Сначала программа получает текст первой страницы документа и разбивает его на отдельные строки. Дополнительно сохраняются координаты текста и размер шрифта.

```

for block in page_dict["blocks"]:
    if block.get("type") != 0:
        continue

```

Это позволяет передавать в модель не только текст, но и информацию о расположении строк внутри страницы.

После извлечения строки очищаются от лишних символов и номеров сносок. Для этого используется функция `clean_metadata_line()`.

```

def clean_metadata_line(text):
    text = clean_text(text)
    text = re.sub(r'([А-Яа-яЁёА-Zа-з])\d+$', r'\1', text)
    return text.strip()

```

Далее программа формирует список строк, которые будут отправлены в нейросеть.

```

useful_lines.append({
    "line_number": index,
    "text": line["text"],
    "font_size": line["font_size"],
    "x": line["x"],
    "y": line["y"]
})

```

После подготовки данных вызывается функция `ask_qwen_for_metadata()`, которая формирует `prompt` и отправляет его в модель.

```
def ask_qwen_for_metadata(lines):
    lines_text = json.dumps(lines, ensure_ascii=False, indent=2)

    prompt = f"""Ты извлекаешь метаданные русскоязычной научной статьи. Тебе
дан список строк с первой страницы PDF. У каждой строки есть: line_number- text-
font_size- x- y. Нужно найти: 1. Название статьи. 2. Автора или авторов статьи.
Очень важные правила: - Верни только JSON. - Не придумывай данные. - Бери данные
только из переданных строк. - Автор — это человек: фамилия и инициалы. -
Примеры автора: "И. А. Горбунов", "Д. В. Зайцев", "Зайцев Д. В.", "И. Е. Малова". -
Название статьи НЕ может состоять только из имени автора. - Если строка
выглядит как "Д. В. Зайцев", это автор, а не название. - Название статьи обычно
находится рядом с автором: выше или ниже. - Название может быть написано
заглавными буквами. - Не бери название журнала. - Не бери название
университета. - Не бери слова "Аннотация" и "Ключевые слова". - Не бери
должности, степени и звания как часть имени автора. - Если в строке есть автор,
запятая и должность, оставь только имя автора. - authors должен быть списком
строк, а не списком объектов. - Если первая строка самая крупная и не похожа на
имя человека, почти всегда это название статьи. - Если следующая строка похожа
на ФИО, это автор. - Удали сноски из названия, например "логики1" ->
"логики". Верни строго в таком формате: {{ "title": "", "authors": [] }}
Строки первой
страницы: {lines_text}"""
    try:
        response = requests.post(
            OLLAMA_URL,
            json={
                "model": OLLAMA_MODEL,
                "prompt": prompt,
                "stream": False,
                "format": "json",
                "options": {
                    "temperature": 0
                }
            },
            timeout=180
        )

        if response.status_code != 200:
            print("Ошибка Ollama:", response.status_code)
            print(response.text)

        return {
            "title": "",
            "authors": []
        }
```

```

answer = response.json()["response"].strip()
data = json.loads(answer)

title = data.get("title", "")
authors = normalize_authors(data.get("authors", []))

return {
    "title": title.strip(),
    "authors": authors
}

except Exception as error:
    print("Ошибка при обращении к Qwen:", error)

return {
    "title": "",
    "authors": []
}

```

В prompt дополнительно указываются правила, помогающие модели отличать название статьи от авторов. Например, объясняется, что автор обычно содержит фамилию и инициалы, а название статьи не должно выглядеть как имя человека.

После получения ответа программа извлекает название статьи и авторов, а затем сохраняет их вместе с найденными теоремами и леммами.

### 3.5. Формирование итогового датасета

После обработки PDF-документов программа формирует итоговый JSON-файл с найденными данными. Для каждой найденной теоремы или леммы создаётся отдельная запись.

В итоговый датасет сохраняются:

- имя PDF-файла;
- название статьи;
- авторы;

- номер страницы;
- тип найденного фрагмента;
- текст теоремы или леммы;
- начальный и конечный индексы.

Добавление данных выполняется через список `all_results`.

```
all_results.append({
    "file": file,
    "article_title": article_title,
    "article_authors": ", ".join(article_authors),
    "page": page_num + 1,
    "type": block["type"],
    "text": block["text"],
    "start_index": block["start_index"],
    "end_index": block["end_index"]
})
```

После завершения обработки всех файлов результат сохраняется в файл `result.json`.

```
with open("result.json", "w", encoding="utf-8") as f:
```

Для записи данных используется функция `json.dump()`.

```
json.dump(
    all_results,
    f,
    ensure_ascii=False,
    indent=4)
```

В результате формируется структурированный JSON-файл, содержащий найденные теоремы и леммы вместе с информацией о научных статьях.

### 3.6. Анализ результатов работы программы

После запуска программы был сформирован файл result.json, содержащий результаты обработки PDF-документов. В итоговый датасет попали сведения о статье, название, авторы, номер страницы и найденные теоремы или леммы.

Пример структуры сохранённых данных выглядит следующим образом.

```
"file": "elibrary_16968606_93799264.pdf",  
"article_title": "Хорошо определенные логики",  
"article_authors": "И. А. Горбунов",  
"page": 3,  
"type": "theorem",  
"text": "ТЕОРЕМА 1. Если  $\alpha \in C(X)$ , то  $C(\alpha, X) = C(X)$ .",  
"start_index": 1706,  
"end_index": 1749
```

По результатам работы можно сделать вывод, что программа корректно выполняет извлечение теорем и лемм из научных PDF-документов. Также удалось реализовать автоматическое определение названия статьи и авторов при помощи локальной нейросети.

Во время тестирования программы были выявлены некоторые сложности, связанные со структурой PDF-документов. В отдельных статьях встречались сноски, нестандартное расположение текста и ошибки при извлечении формул. Для решения этих проблем была добавлена дополнительная очистка текста и обработка строк с метаданными.

Полученный датасет можно использовать для дальнейшего анализа математических текстов, разметки данных или последующего обучения моделей для обработки научных публикаций.

### 3.7. Вывод по разделу

В ходе разработки была создана программа для автоматической обработки научных PDF-документов и формирования датасета. Программа выполняет чтение PDF-файлов, очистку текста, поиск теорем и лемм, а также сохранение результатов в JSON-формате.

Дополнительно была реализована работа с локальной нейросетевой моделью через Ollama для определения названия статьи и авторов. Полученный



датасет может использоваться для дальнейшего анализа научных текстов и последующей обработки данных.

## 4. Заключение

В ходе прохождения производственной практики была разработана программа для автоматической обработки научных PDF-документов и формирования датасета с теоремами и леммами.

Во время выполнения работы были изучены особенности структуры научных PDF-файлов, методы извлечения текста и способы обработки математических публикаций при помощи Python. Также были получены практические навыки работы с библиотекой PyMuPDF и локальными нейросетевыми моделями.

В процессе разработки была реализована очистка текста, поиск теорем и лемм при помощи регулярных выражений, а также автоматическое определение названия статьи и авторов через модель Qwen, подключённую локально через Ollama.

Дополнительно была реализована обработка ошибок и очистка метаданных статьи, что позволило повысить качество итогового датасета.

Результатом работы стал JSON-файл, содержащий найденные теоремы и леммы вместе с информацией о научных статьях. Полученный датасет может использоваться для дальнейшего анализа математических текстов, разметки данных и последующей обработки научных публикаций.

Таким образом, цель производственной практики была достигнута. В ходе работы были закреплены теоретические знания и получены практические навыки разработки программ для обработки научной информации.